El Backend en Python, explicado fácil

Graficadora de Transformación de Funciones

Qué hace cada archivo · cómo se conectan · para qué sirve cada parte

$$g(x) = a \cdot f(\, b\,(x - c)\,) + d$$

Idea en una frase: el backend es el **cerebro** del proyecto. Toda la matemática ocurre aquí, en Python. La página web (el frontend) solo **pregunta** y **dibuja** lo que el cerebro responde.

El backend está dividido en **5 archivos** con una regla simple: cada archivo tiene **un solo trabajo**. Así el código es ordenado, fácil de entender y fácil de arreglar. Esto es lo que el enunciado llama "estructura modular".

Mapa de la carpeta

```
backend/
├── app.py              ← la puerta de entrada (servidor web Flask)
├── requirements.txt    ← lista de lo que hay que instalar (Flask)
└── motor/              ← el "cerebro" matemático (un paquete de Python)
    ├── funciones.py        ← QUÉ funciones existen (x², √x, aˣ…)
    ├── transformaciones.py ← CÓMO se transforma y se describe
    └── validacion.py       ← LIMPIA y revisa lo que escribe el usuario
```

En simple: piense en un restaurante. **app.py** es el mesero (recibe el pedido y trae el plato). **validacion.py** revisa que el pedido tenga sentido. **funciones.py** es la despensa (los ingredientes base). **transformaciones.py** es el cocinero (prepara el plato con esos ingredientes).

1 app.py — el mesero (servidor web)

`backend/app.py`

*Para qué sirve: es la **puerta de entrada**. Levanta un pequeño servidor web (con Flask) que escucha peticiones del navegador y responde con los resultados.*

Hace tres cosas, cada una en una "ruta" (una dirección web):

- `GET /` → entrega la página `index.html` (el frontend). Es lo que ve el usuario al abrir `http://127.0.0.1:5000`.
- `GET /api/funciones` → devuelve la **lista de funciones** disponibles para llenar el menú desplegable.
- `POST /api/graficar` → recibe los parámetros del usuario (`funcion, a, b, c, d…`), le pide al cerebro que calcule y devuelve **todo listo**: los puntos de las curvas, la tabla, el resumen y la fórmula.

> **En simple:** es el mesero que recibe tu pedido, se lo pasa a la cocina y te trae el plato servido. No cocina él; solo coordina.

Cómo pide ayuda a los demás archivos: arriba tiene la línea `from motor import transformaciones, validacion, funciones`. Eso significa "voy a usar esos tres ayudantes". Cuando llega un `POST /api/graficar`:

1. Le pasa los datos crudos a `validacion` para limpiarlos.
2. Con los datos ya limpios, le pide a `transformaciones` las curvas, la tabla, el resumen y la fórmula.
3. Empaqueta todo en un **JSON** (un formato de texto que el navegador entiende) y lo devuelve.

> **Detalle de robustez:** si la validación encuentra un error grave, `app.py` responde con código `400` y un mensaje claro, en vez de reventar.

`backend/requirements.txt`

*Para qué sirve: es la **lista de la compra** del proyecto.*

Contiene una sola línea: `Flask==3.0.3`. Le dice a quien quiera correr el proyecto: "instala Flask y listo". Se instala con `pip install -r requirements.txt`. No necesita nada más: toda la matemática está hecha con Python puro.

2 motor/funciones.py — la despensa

`backend/motor/funciones.py`

*Para qué sirve: guarda el **catálogo de funciones base** (las "funciones madre" sin transformar) y sabe cuál es el dominio de cada una.*

Tiene un gran diccionario llamado `FUNCIONES_BASE` con **6 funciones** (el proyecto pedía al menos 5):

- **Línea** $f(x) = x$
- **Cuadrática** $f(x) = x^2$
- **Cúbica** $f(x) = x^3$
- **Valor absoluto** $f(x) = |x|$
- **Raíz cuadrada** $f(x) = \sqrt{x}$ *(solo existe si $x \geq 0$)*
- **Exponencial** $f(x) = a^x$ *(con base $a > 0$ y $a \neq 1$)*

De cada función guarda cuatro cosas: su **nombre** legible, su **fórmula** en texto, la **función matemática** en sí, y su **dominio** (qué valores de x son válidos).

> **En simple:** es la despensa con los ingredientes básicos. Nadie cocina aquí; solo se guardan ordenados y etiquetados, listos para usar.

Las tres herramientas que ofrece

- `listar_funciones()` → entrega la lista corta (clave, nombre, fórmula). La usa `app.py` para llenar el menú del frontend.
- `info_funcion(clave)` → entrega todos los datos de una función (incluido su dominio). La usa `transformaciones.py`.
- `construir_funcion_base(clave, base_exp)` → devuelve la función lista para calcular. La exponencial es especial: necesita una base (por defecto 2), y aquí mismo se revisa que la base sea válida (> 0 y $\neq 1$).

> **Por qué importa el dominio:** la raíz cuadrada de un número negativo no existe. Guardar el dominio aquí permite que el resto del programa evite ese error automáticamente.

Se conecta con: lo llaman `transformaciones.py` (para calcular) y `validacion.py` (para confirmar que la función elegida existe).

3 motor/transformaciones.py — el cocinero

`backend/motor/transformaciones.py`

*Para qué sirve: es el **corazón matemático**. Aplica el modelo* `g(x) = a · f(b(x − c)) + d` *y produce todo lo que se ve en pantalla.*

Toma una función de la despensa y le aplica los cuatro parámetros. Recuerde qué hace cada uno:

- **a** → estira o encoge *a lo alto* (y si es negativo, la voltea sobre el eje X).
- **b** → estira o encoge *a lo ancho* (y si es negativo, la voltea sobre el eje Y).
- **c** → la mueve a la *derecha o izquierda*.
- **d** → la mueve *arriba o abajo*.

> **En simple:** es el cocinero. Toma el ingrediente base (la función) y, siguiendo la receta (a, b, c, d), lo transforma en el plato final (la gráfica, la tabla y la explicación).

Las cinco herramientas que ofrece

- `evaluar_g(f, x, a, b, c, d)` → calcula el valor de g en **un** punto. Si el punto cae fuera del dominio (ej. raíz de negativo), devuelve `None` (nada) en vez de fallar.
- `muestrear(...)` → genera **400 puntos** entre x mínimo y x máximo y calcula f(x) y g(x) en cada uno. Esto es lo que el frontend usa para **dibujar las dos curvas**. Donde un punto no existe, deja un hueco (no inventa línea).
- `tabla_puntos(...)` → arma la **tabla** de valores (x, f(x), g(x)) con al menos 5 filas, redondeados para que se lean bien.
- `resumen_interpretativo(...)` → genera la **explicación en español**, frase por frase: “a = 2: estiramiento vertical por factor 2”, “c = 1: traslación a la derecha”, etc. Usa condicionales (`if`) para describir solo lo que de verdad pasó.
- `formula_g(...)` → escribe la **fórmula final ya con números**, por ejemplo `g(x) = 2·(3(x − 1))² + 4`.

> **Dato clave de diseño:** este archivo *no sabe nada* de HTML, de Flask ni de colores. Solo hace matemática. Por eso es reutilizable: podría conectarse a otra interfaz sin cambiar una línea.

Se conecta con: usa a `funciones.py` (le pide la función y su dominio) y es llamado por `app.py` (que le pide los resultados).

4 motor/validacion.py — el portero

`backend/motor/validacion.py`

*Para qué sirve: **revisa y limpia** lo que escribe el usuario antes de que llegue a la matemática. Es la red de seguridad del proyecto.*

El usuario puede equivocarse: dejar una casilla vacía, escribir "hola" donde va un número, o poner un intervalo al revés. Este archivo se encarga de que **nada de eso rompa el programa**.

> **En simple:** es el portero de la discoteca. Revisa a cada dato en la entrada: al que viene mal vestido (un texto donde iba un número) lo corrige o lo manda de vuelta, para que adentro todo funcione en orden.

Las dos herramientas que ofrece

- `a_numero(valor, defecto)` → intenta convertir el dato a número. Si viene vacío o es texto, usa un **valor por defecto** en vez de reventar. Así se cumple el requisito de "no permitir valores alfanuméricos donde van números".
- `validar_parametros(datos)` → recibe todo el pedido crudo y devuelve una versión **limpia y segura**. Aplica los valores por defecto (`a=1, b=1, c=0, d=0`), revisa que la función exista, valida la base de la exponencial, corrige el intervalo si está al revés (x_min ≥ x_max) y garantiza mínimo 5 puntos en la tabla.

> **Dos niveles de respuesta:** los errores leves se corrigen solos (con valores por defecto); los graves de verdad —como elegir una función que no existe— lanzan un aviso que `app.py` convierte en un mensaje de error limpio para el usuario.

Se conecta con: usa a `funciones.py` (para confirmar que la función existe) y es llamado por `app.py` como **primer paso**, antes de calcular nada.

5 Cómo se conecta todo: un pedido de principio a fin

Esto es lo que pasa, paso a paso, desde que el usuario mueve un parámetro hasta que ve la gráfica nueva:

1. El usuario, en la **página web**, elige una función y pone `a, b, c, d`. El navegador manda esos datos a `POST /api/graficar`.
2. **app.py** recibe el pedido y, antes que nada, se lo pasa al portero: `validacion.validar_parametros(...)`.
3. **validacion.py** limpia los datos (números válidos, defaults, intervalo coherente) y, de paso, le pregunta a **funciones.py** si la función elegida existe. Devuelve un paquete limpio.
4. Con los datos limpios, **app.py** llama al cocinero **transformaciones.py** cuatro veces: curvas (`muestrear`), tabla (`tabla_puntos`), explicación (`resumen_interpretativo`) y fórmula (`formula_g`).
5. **transformaciones.py** toma la función base desde **funciones.py**, aplica `g(x) = a·f(b(x − c)) + d` y devuelve todos los resultados ya calculados.
6. **app.py** junta todo en un **JSON** y se lo devuelve al navegador.
7. El **frontend** recibe ese JSON y dibuja: las dos curvas, la tabla, el resumen y la fórmula. *El navegador no calculó nada: solo pintó la respuesta de Python.*

Las tres ideas para recordar

- **Separación de tareas:** cada archivo hace una sola cosa (servir, validar, guardar funciones, calcular). Eso es la “estructura modular” del enunciado.
- **Python piensa, la web muestra:** toda la matemática está en Python; el frontend solo pide y dibuja. Por eso se cumple el requisito de “hecho en Python” aun siendo una página web.
- **A prueba de errores:** el programa no se cae con datos malos; los corrige o avisa con claridad.